

UNIVERSIDAD DE ESPECIALIDADES ESPÍRITU SANTO

Facultad de Sistemas, Telecomunicaciones y Electrónica · Computación

Reporte técnico

Refactorización avanzada respaldada por pruebas unitarias

Sistema de gestión de tutorías académicas · Actividad evaluada Ae5

Asignatura	Diseño de Software — UCOM0310
Actividad	Ae5 Refactorización respaldada por pruebas unitarias (Semana 6)
Periodo	PEL 4 — 2026
Estudiante	Luis Saá
Docente	Ph.D. Jaime Paul Sayago Heredia
Fecha de entrega	21 de septiembre de 2026
Repositorio	https://github.com/saaluis2242-droid/sistema-tutorias (rama main)
Entregable	Código inicial y final, pruebas JUnit 5, seis refactorizaciones, historial Git, evidencias de ejecución y este reporte

Pregunta central de la actividad: ¿cómo puedo demostrar que el diseño mejoró sin modificar accidentalmente el comportamiento que debía preservarse? *La respuesta de este trabajo es una red de pruebas de caracterización construida antes de tocar el diseño, seis refactorizaciones aplicadas de una en una, y un historial Git donde cada commit registra el estado de la suite: 80 pruebas, verdes en los siete pasos, sin una sola aserción modificada para acomodarla al nuevo diseño.*

1. Descripción del problema inicial

El proyecto integrador de la asignatura es un sistema de gestión de tutorías académicas: un docente publica bloques de horario disponibles, un estudiante solicita una tutoría para uno de esos bloques y la reserva resultante puede confirmarse, cancelarse o reprogramarse, notificando a los involucrados y generando el recibo correspondiente.

El código de partida de Ae5 es el resultado acumulado de cuatro entregas previas: el modelo orientado a objetos de Ae1, la práctica de patrones creacionales de Ae2, la integración de Strategy y Observer de Ae3 y la kata de refactorización de Ae4. Ese recorrido dejó un diseño razonable en su arquitectura —persistencia, política de cancelación y canal de notificación están detrás de abstracciones— pero también un conjunto de problemas de diseño que las entregas anteriores no alcanzaron a atacar, porque Ae4 trabajó deliberadamente sobre una clase pequeña y aislada.

Ae5 se plantea entonces sobre el código heredado del propio sistema, no sobre un ejemplo autocontenido: clases con más de una razón para cambiar, tipos primitivos cargando reglas de negocio, reglas del dominio validadas desde la capa de aplicación y un grupo de datos que viaja siempre junto pero sin nombre propio.

2. Línea base

El punto de partida quedó fijado en el primer commit del historial de esta entrega, para que cualquier comparación posterior sea verificable:

Elemento	Estado al iniciar Ae5
Commit de línea base	5b9e3b1 — «chore: línea base Ae4 (punto de partida de Ae5)»
Código de producción	20 clases en 8 paquetes (domain, service, service.cancelacion, repository, notification, notification.factory, notification.observer, reporte, app)
Pruebas automatizadas	10 pruebas JUnit 5 en 2 clases (ServicioReservasTest, NotificadorCreatorFactoryTest) más un harness manual sin JUnit (LineaBaseRecibos, 6 casos)
Patrones vigentes	Strategy (política de cancelación), Observer (reacciones a cambios de estado), Factory Method (canal de notificación)
Herramientas	Java 21, Maven 3.9, JUnit 5.10.2, Surefire 3.2.5
Estado de la suite	Verde antes de iniciar cualquier cambio

Restricción del entorno y cómo se resolvió. El entorno donde se prepararon las refactorizaciones no tiene acceso de red a Maven Central: `repo.maven.apache.org` responde 403 por política de egreso, de modo que no fue posible descargar junit-jupiter ni Surefire y `mvn clean test` no puede resolver sus dependencias. Para no renunciar al ciclo obligatorio de verificación, se construyó `tools/verificar-offline.sh`: compila las mismas pruebas JUnit 5 del proyecto contra stubs mínimos de la API (`@Test`, `@BeforeEach`, `Assertions`) y las ejecuta con un ejecutor propio por reflexión que respeta el aislamiento por instancia de JUnit y devuelve código de salida 1 si alguna prueba falla. Las pruebas no se modificaron para este ejecutor y los stubs viven fuera de `src/`, por lo que la ejecución oficial con `mvn clean test` las compila contra el JUnit real. Cada log de «suite verde» de este reporte proviene de esa ejecución real, no de una revisión a mano.

3. Diagnóstico resumido

Los problemas atacados en esta entrega se seleccionaron por dos criterios: que tuvieran una consecuencia concreta y demostrable sobre el mantenimiento (no una preferencia estética), y que estuvieran cubiertos o fueran cubribles por pruebas antes de tocarlos.

Ubicación	Problema de diseño	Consecuencia concreta
Usuario	Primitive Obsession: el correo es un String validado por un método privado	El tipo no comunica la regla; cualquier método que reciba un "String correo" no puede saber si ya fue validado, y la regla tiende a duplicarse en cada punto de entrada
Docente	Dos razones de cambio: identidad del usuario y administración de la agenda con su regla de no solapamiento	Toda evolución de la disponibilidad (cupos, retiro de horarios, próximo libre) engorda una clase cuya responsabilidad es representar a una persona
ServicioReservas	Reglas del dominio validadas en la capa de aplicación: pregunta isDisponible() y luego llama a marcarOcupado()	La pregunta y la acción quedan separadas: otro cliente puede omitir la pregunta u operar sobre un estado ya cambiado; reprogramar por otra vía saltea la validación
ServicioReservas	Feature Envy: arma la nota de cancelación interrogando los datos de ResultadoCancelacion con un ternario sin nombre	Un cambio de formato de la nota obliga a tocar el servicio; la condición no dice qué representa
Cadena solicitarReserva → Reserva	Data Clump: el trío (estudiante, docente, horario) viaja junto y suelto por cada firma	Dos de los tres parámetros son Usuario, así que invertirlos compila sin queja; la validación está repetida y un cuarto dato cambiaría todas las firmas
GeneradorReciboReserva	Dos razones de cambio: decide cuánto se cobra y cómo se ve el recibo; el precio circula como double suelto	Cambiar un descuento y agregar una línea al recibo llegan al mismo archivo; la regla de precio solo puede probarse leyendo el texto impreso

4. Pruebas utilizadas como red de seguridad

El primer commit de trabajo de Ae5 no cambia ni una línea de código de producción: amplía la suite hasta 54 pruebas que describen lo que el sistema hace hoy —no lo que debería hacer—, incluidos los mensajes exactos de cada error y el texto de los recibos carácter por carácter. Ese es el criterio que permite afirmar después que el comportamiento se preservó.

Clase de prueba	Comportamiento que fija	Refactorización que protege
GeneradorReciboReservaTest	Los 6 recibos del harness de Ae4 migrados a JUnit 5, comparados carácter por carácter, con el formato del total y los dos caminos del descuento de fidelidad	R6
UsuarioTest	Validación de nombre y correo con sus mensajes exactos, canal por defecto, igualdad por id, rol de cada subtipo	R1

Clase de prueba	Comportamiento que fija	Refactorización que protege
HorarioTest	Validación de inicio/fin, ocupar y liberar, solapamiento, igualdad por intervalo y el toString() del que depende el recibo	R3, R6
ReservaTest	Todas las transiciones válidas e inválidas con sus mensajes, y el efecto de cada una sobre el horario y las notas	R3, R5
DocenteTest	Publicación de horarios, orden, rechazo del solapamiento y colección de solo lectura	R2
ServicioReservasCaracterizacionTest	Mensajes exactos de cada rechazo, composición de la nota de cancelación, orden de los eventos publicados y ausencia de efectos a medias cuando una operación se rechaza	R3, R4, R5
ServicioReservasTest (de Ae4)	Flujo completo del caso de uso. Se mantuvo sin modificar, como control independiente	Todas

Ciclo de trabajo aplicado, sin excepciones: *prueba verde* → *cambio pequeño* → *prueba verde* → *commit* → *siguiente cambio*. Ninguna refactorización se acumuló con otra y la suite completa se ejecutó antes y después de cada cambio. Los logs por commit se regeneraron ejecutando la suite sobre cada commit del historial, de modo que cada archivo de evidencia corresponde exactamente a ese estado del código.

5. Refactorización 1 — Introduce Value Object (Correo)

Problema de diseño	Usuario guardaba el correo como String y validaba su forma en el método privado validarCorreo (Primitive Obsession). El tipo del atributo no comunicaba ninguna regla, la validación vivía lejos del dato y cualquier clase que recibiera un "String correo" no podía saber si ya estaba validado.
Código antes	<pre>private final String correo; / private String validarCorreo(String correo) { if (correo == null !correo.contains("@")) throw new IllegalArgumentException(...); return correo; }</pre>
Técnica aplicada	Introduce Value Object: record Correo con validación en el constructor compacto, inmutable e igual por valor.
Prueba que protege	UsuarioTest.unCorreoSinArrobaEsRechazadoConElMismoMensajeDeSiempre, unCorreoNuloEsRechazado, elCorreoSeLeeDeVueltaComoElMismoTexto; NotificadorCreatorFactoryTest (la línea de email sigue componiéndose igual).
Resultado después	Un Correo no puede existir inválido: si hay instancia, es válida por construcción, y el tipo lo declara en cada firma donde aparece. Usuario dejó de conocer qué hace válido a un correo. Se agregó CorreoTest, que prueba la regla sin construir un Usuario completo.
Commit	d25dffc — «refactor: introduce el Value Object Correo en lugar de un String validado». Suite: 58/58 verde.

```
// después
public record Correo(String valor) {
    public Correo {
        if (valor == null || !valor.contains("@")) {
            throw new IllegalArgumentException("El correo del usuario no es valido");
        }
    }
    @Override public String toString() { return valor; }
}
```

El toString() devuelve el texto original, de modo que los notificadores que interpolan el correo imprimen exactamente lo mismo que antes: el comportamiento observable no cambió.

6. Refactorización 2 — Extract Class (AgendaDocente)

Problema de diseño	Docente tenía dos razones de cambio conviviendo: era un Usuario (identidad, nombre, correo, canal preferido, rol) y al mismo tiempo administraba una estructura de datos con su propia regla de negocio (lista de horarios, validación de solapamiento, exposición de solo lectura).
Código antes	<pre>private final List<Horario> horarios = new ArrayList<>(); / public void publicarHorario(Horario h) { boolean seSolapa = horarios.stream().anyMatch(x -> x.seSolapaCon(h)); if (seSolapa) throw ...; horarios.add(h); }</pre>
Técnica aplicada	Extract Class + Move Method: la colección y la regla pasan a AgendaDocente; Docente

	delega y conserva su interfaz pública.
Prueba que protege	DocenteTest.losHorariosSePublicanYSeConservanEnOrden, unHorarioQueSeSolapaConOtroYaPublicadoEsRechazado (con el mensaje exacto), dosHorariosConsecutivosNoSeConsideranSolapados, laAgendaExpuestaEsDeSoloLectura.
Resultado después	Cada clase con una razón de cambio: Docente representa a la persona, AgendaDocente es el lugar natural donde crecen las reglas de disponibilidad. Ningún cliente cambió (Main, ServicioReservas, pruebas), porque la interfaz pública de Docente se mantuvo. Se agregó AgendaDocenteTest, que prueba la regla sin construir un Docente.
Commit	fcab73f — «refactor: extrae AgendaDocente de Docente (Extract Class)». Suite: 62/62 verde.

Costo asumido y registrado: al mover el método se detectó que no valida el nulo (publicar(null) sobre una agenda vacía lo agregaría a la lista). El defecto se dejó tal como estaba a propósito, porque una refactorización no debe corregir comportamiento; queda documentado en el javadoc de AgendaDocente como cambio funcional pendiente, con su propia prueba, para un commit que lo declare como tal.

7. Refactorización 3 — Move Method + Guard Clauses

Problema de diseño	ServicioReservas validaba reglas que pertenecen al dominio. En solicitarReserva preguntaba horario.isDisponible() y, si la respuesta era afirmativa, llamaba a horario.marcarOcupado(): pregunta y acción separadas, con el servicio conociendo una regla interna del Horario. En reprogramarReserva comprobaba la disponibilidad del nuevo horario antes de delegar en Reserva.reprogramar, de modo que reprogramar por otra vía salteaba la validación.
Código antes	<pre>if (!horario.isDisponible()) { throw new IllegalStateException("El horario seleccionado ya no esta disponible"); } horario.marcarOcupado();</pre>
Técnica aplicada	Move Method (la regla al objeto que posee el dato) y Guard Clauses (validar todo al inicio, mutar solo después).
Prueba que protege	ServicioReservasCaracterizacionTest.solicitarUnHorarioOcupadoFallaConSuMensajeYSi nPublicarEventos y reprogramarHaciaUnHorarioOcupadoFallaConSuMensajeYNoAlteraLaReserva; ServicioReservasTest.noSePuedeReservarUnHorarioYaOcupado (suite de Ae4).
Resultado después	Horario.reservar() valida y ocupa en una operación indivisible, con el mismo mensaje que emitía el servicio. Reserva.reprogramar() concentra sus tres validaciones como guard clauses y no muta nada antes de que todas pasen: antes, un nuevo horario nulo liberaba el horario anterior y recién después fallaba, dejando la reserva a medio camino. ServicioReservas perdió ambos bloques condicionales y quedó reducido a orquestación.
Commit	7902476 — «refactor: mueve la regla de disponibilidad al dominio con guard clauses». Suite: 66/66 verde.

```
// después – en Horario
public void reservar() {
    if (!disponible) {
        throw new IllegalStateException("El horario seleccionado ya no esta disponible");
    }
    this.disponible = false;
}

// después – en ServicioReservas
horario.reservar();
Reserva reserva = new Reserva(estudiante, docente, horario);
```

8. Refactorización 4 — Decompose Conditional + Move Method

Problema de diseño	ServicioReservas.cancelarReserva armaba la nota con un operador ternario que preguntaba resultado.getObservacion().isEmpty() y concatenaba corchetes según la respuesta: Feature Envy —el servicio interrogaba los datos de otro objeto para decidir en su lugar— y una condición sin nombre mezclada con la orquestación del caso de uso.
Código antes	String notaFinal = resultado.getObservacion().isEmpty() ? motivo : motivo + " [" + resultado.getObservacion() + "]; reserva.cancelar(notaFinal);
Técnica aplicada	Decompose Conditional (nombrar la condición) + Move Method (llevar la decisión al objeto que posee el dato).
Prueba que protege	ServicioReservasCaracterizacionTest.laObservacionDeLaPoliticaSeAnexaAlMotivoEntre Corchetes y sinObservacionLaNotaEsExactamenteElMotivo; ServicioReservasTest.cancelarReservaTardiaQuedaMarcadaPorLaPoliticaConAntelacion (suite de Ae4).
Resultado después	ResultadoCancelacion.componerNota(motivo) concentra la regla, con la condición nombrada tieneObservacion(). El servicio solo pide la nota y su método quedó en cinco pasos de orquestación: obtener, evaluar, cancelar, guardar, publicar. Se agregó ResultadoCancelacionTest, que prueba el formato de la nota sin montar repositorio, política ni servicio.
Commit	41f14c7 — «refactor: mueve la composicion de la nota de cancelacion a ResultadoCancelacion». Suite: 69/69 verde.

9. Refactorización 5 — Agrupar Data Clump (SolicitudTutoria)

Problema de diseño	El trío (estudiante, docente, horario) viajaba siempre junto y siempre suelto: el cliente llamaba a solicitarReserva(estudiante, docente, horario), que reenviaba los mismos tres argumentos en el mismo orden a new Reserva(estudiante, docente, horario). Dos de los tres parámetros son Usuario, así que invertirlos compilaba sin queja; la validación del trío estaba repetida; y agregar un cuarto dato de la solicitud (modalidad, asignatura, motivo) habría obligado a cambiar todas las firmas de la cadena.
Código antes	<pre>public Reserva solicitarReserva(Estudiante estudiante, Docente docente, Horario horario) { ... new Reserva(estudiante, docente, horario); }</pre>
Técnica aplicada	Agrupar Data Clump / Introduce Parameter Object: record SolicitudTutoria que agrupa y valida los tres datos una sola vez.
Prueba que protege	Toda la suite de ServicioReservas y ReservaTest, que siguen usando las firmas de tres argumentos y por lo tanto verifican que la delegación no cambió nada; ServicioReservasCaracterizacionTest.unaSolicitudIncompletaNoConsumeElHorario.
Resultado después	La solicitud es ahora el vocabulario del caso de uso y se valida en un único lugar. Las firmas anteriores se conservaron como sobrecargas que delegan, para no romper ningún cliente. Efecto colateral positivo: como la solicitud se valida antes de tocar el horario, una solicitud incompleta ya no deja el bloque ocupado e inutilizable —antes, marcarOcupado() se ejecutaba primero y el fallo posterior del constructor dejaba el horario perdido—. Se agregó SolicitudTutoriaTest.
Commit	451cd1a — «refactor: agrupa el data clump estudiante/docente/horario en SolicitudTutoria». Suite: 74/74 verde.

```
// después
public record SolicitudTutoria(Estudiante estudiante, Docente docente, Horario horario) {
    public SolicitudTutoria {
        Objects.requireNonNull(estudiante, "El estudiante es obligatorio");
        Objects.requireNonNull(docente, "El docente es obligatorio");
        Objects.requireNonNull(horario, "El horario es obligatorio");
    }
}
```

10. Refactorización 6 — Extract Class + Value Object (TarifarioTutoria, Dinero)

Problema de diseño	GeneradorReciboReserva cargaba con dos razones de cambio: decidía cuánto se cobra (PRECIO_BASE, DESCUENTO_REPROGRAMACION, DESCUENTO_FIDELIDAD, el prefijo del código institucional y el switch que los aplica) y además cómo se ve el recibo (rótulos, orden de las líneas, saltos). Que el área académica cambie un descuento y que se agregue una línea al recibo son cambios sin relación, y los dos llegaban al mismo archivo. Además el precio circulaba como un double suelto, con su formato escrito a mano en la concatenación final y la expresión precio - (precio * descuento) repetida en dos ramas.
---------------------------	--

Código antes	<pre>private static final double PRECIO_BASE = 15.0; ... recibo = recibo + "Total a pagar: \$" + precio + "\n";</pre>
Técnica aplicada	Extract Class (la regla de cobro) + Introduce Value Object (el importe).
Prueba que protege	GeneradorReciboReservaTest: los 6 recibos comparados carácter por carácter, incluidos los totales \$15.0, \$13.5, \$0.0 y \$12.0. Adicionalmente, el harness manual de Ae4 se re-ejecutó contra el código final y su diff contra la salida capturada en Ae4 no reporta ninguna diferencia.
Resultado después	TarifarioTutoria concentra la regla de cobro y devuelve un importe; Dinero lleva su única operación de negocio (aplicar un descuento) y su propio formato; el generador quedó como formateador y admite recibir otro tarifario por constructor (por ejemplo, una tarifa promocional o una de prueba). Se agregó TarifarioTutoriaTest, que verifica cada regla de cobro sin pasar por el texto del recibo.
Commit	3f052d6 — «refactor: extrae TarifarioTutoria y el Value Object Dinero del generador de recibos». Suite: 80/80 verde.

Costo asumido y registrado: Dinero conserva double como representación interna y la expresión aritmética exacta que ya existía, para que el total no cambie ni en el último decimal. Migrar a BigDecimal sería lo correcto para dinero, pero es un cambio de comportamiento y no una refactorización; queda declarado como trabajo posterior en el javadoc de la clase.

11. Resultado final de las pruebas

La suite pasó de 54 a 80 pruebas y quedó verde en los siete commits de trabajo, sin una sola prueba roja y sin modificar ninguna aserción existente para acomodarla al nuevo diseño:

Paso	Commit	Pruebas	Resultado	Evidencia
Red de seguridad	82f82ff	54 / 54	SUITE VERDE	01-red-de-seguridad-verde.txt
R1 — Value Object Correo	d25dff	58 / 58	SUITE VERDE	02-refactorizacion-1-value-object-correo.txt
R2 — Extract Class AgendaDocente	fcab73f	62 / 62	SUITE VERDE	03-refactorizacion-2-extract-class-agenda.txt
R3 — Move Method + Guard Clauses	7902476	66 / 66	SUITE VERDE	04-refactorizacion-3-move-method-guard-clauses.txt
R4 — Decompose Conditional	41f14c7	69 / 69	SUITE VERDE	05-refactorizacion-4-decompose-conditional.txt
R5 — Data Clump SolicitudTutoria	451cd1a	74 / 74	SUITE VERDE	06-refactorizacion-5-data-clump-solicitud.txt
R6 — Extract Class Tarifario + Dinero	3f052d6	80 / 80	SUITE VERDE	07-refactorizacion-6-extract-class-tarifario.txt
Estado final del repositorio	HEAD	80 / 80	SUITE VERDE	08-suite-verde-estado-final.txt

```
$ bash tools/verificar-offline.sh (estado final)
-----
Pruebas ejecutadas: 80, correctas: 80, fallidas: 0
RESULTADO: SUITE VERDE

$ diff docs/ae4-evidencias/linea-despues.txt docs/ae5-evidencias/07-linea-base-recibos-ae5.txt
(sin diferencias: el recibo es idéntico carácter por carácter al de Ae4)
```

Verificación oficial pendiente de entorno: por el bloqueo de red descrito en la sección 2, el reporte oficial de Surefire debe generarse con `bash tools/verificar.sh` en una máquina con acceso normal a internet; el script ejecuta `mvn clean test` y deja la salida completa en `docs/ae5-evidencias/mvn-clean-test.txt`, lista para adjuntar.

Ejecución de la aplicación sobre el código refactorizado: Main se ejecutó contra el diseño final y reprodujo el flujo completo —solicitud, confirmación, reprogramación y cancelación tardía marcada por la política— con las mismas notificaciones por WhatsApp y correo y el mismo registro de auditoría (`docs/ae5-evidencias/09-ejecucion-main.txt`).

12. Comparación antes/después

Dimensión	Antes (cierre de Ae4)	Después (Ae5)	Evidencia
Responsabilidades	Docente era identidad + agenda; GeneradorReciboReserva era tarifa + formato; ServicioReservas orquestaba y además validaba reglas del dominio	Cada clase con una sola razón de cambio: AgendaDocente, TarifarioTutoria y un servicio reducido a orquestación	diff-src-main-ae4-vs-ae5.patch; codigo-antes/ vs codigo-despues/
Cohesión	Reglas de negocio repartidas entre la capa de aplicación y el dominio	Cada regla vive en el objeto que posee el dato: Horario.reservar, Reserva.reprogramar, ResultadoCancelacion.componerNota, TarifarioTutoria.calcular	R3, R4, R6
Acoplamiento	El servicio conocía reglas internas de Horario y el formato de la nota; el generador conocía la política de precios	El servicio depende solo de abstracciones y de operaciones con intención; el generador recibe un tarifario inyectable	R3, R4, R6
Datos del dominio	String correo, double precio y tres parámetros sueltos repetidos por firma	Value Objects Correo y Dinero y el parameter object SolicitudTutoria, válidos por construcción	R1, R5, R6
Condicionales	Ternario de composición de nota en el servicio; pregunta y acción separadas; validaciones intercaladas con mutaciones	Condiciones con nombre, guard clauses al inicio y ninguna mutación antes de validar	R3, R4
Datos del dominio (clases nuevas)	20 clases de producción	25 clases: Correo, AgendaDocente, SolicitudTutoria, Dinero, TarifarioTutoria	diff-resumen-ae4-vs-ae5.txt
Pruebas	10 pruebas JUnit + un harness manual sin JUnit	80 pruebas JUnit en 11 clases; las reglas extraídas se prueban de forma aislada, sin montar el servicio ni leer texto formateado	Logs 01-08 en docs/ae5-evidencias/
Git	1 commit de línea base + 4 de la kata de Ae4	1 de línea base + 1 de red de seguridad + 6 de refactorización + 3 de documentación y evidencias, cada uno con su justificación y el estado de la suite	git-log-ae5.txt, git-log-ae5-detallado.txt

13. Historial Git

El historial es la evidencia de que el proceso fue incremental y controlado: un commit por refactorización, ninguno que acumule dos, y el cuerpo de cada mensaje registra el problema de diseño, el cambio, las pruebas que lo protegen y el estado de la suite tras el cambio.

Commit	Tipo	Mensaje	Suite
5b9e3b1	chore	linea base Ae4 (punto de partida de Ae5)	verde (10)
82f82ff	test	red de seguridad de caracterizacion para Ae5	54 / 54
d25dff	refactor	introduce el Value Object Correo en lugar de un String validado	58 / 58
fcab73f	refactor	extrae AgendaDocente de Docente (Extract Class)	62 / 62
7902476	refactor	mueve la regla de disponibilidad al dominio con guard clauses	66 / 66
41f14c7	refactor	mueve la composicion de la nota de cancelacion a ResultadoCancelacion	69 / 69
451cd1a	refactor	agrupa el data clump estudiante/docente/horario en SolicitudTutoria	74 / 74
3f052d6	refactor	extrae TarifarioTutoria y el Value Object Dinero del generador de recibos	80 / 80
7e8e0be	docs	documenta Ae5 en el README y agrega las evidencias de la entrega	80 / 80
3e01953	docs	guia de commits de Ae5 y evidencias finales de ejecucion	80 / 80
(último)	docs	actualiza el historial Git, los diffs y los parches de la entrega	80 / 80

Los siete commits de trabajo están además exportados como parches aplicables con git am en docs/ae5-evidencias/parches/, de modo que el docente puede reproducir la secuencia paso a paso sobre la línea base y ejecutar la suite en cada punto.

14. Conclusiones

- **La red de pruebas es lo que hace posible la refactorización, no su documentación posterior.** Construir la antes —y como commit separado que no toca código de producción— convirtió cada cambio posterior en una hipótesis verificable en segundos en lugar de una apuesta.
- **Caracterizar incluye los detalles que nadie declara.** Fijar los mensajes exactos de cada excepción y el recibo carácter por carácter fue lo que permitió mover validaciones entre clases con la certeza de que ningún cliente notaría el cambio; una prueba que solo afirmara el tipo de excepción habría dejado pasar un mensaje distinto.
- **Las técnicas resolvieron problemas con consecuencias, no preferencias estéticas.** En cada caso el criterio de selección fue el mismo: cuántas razones de cambio tenía la clase y qué evolución previsible la habría hecho crecer. Ese criterio es también el que sostiene la defensa de cada decisión.
- **La disciplina incluye lo que se decide no hacer.** El defecto latente de publicarHorario y el double de Dinero se dejaron intactos y documentados: mezclar corrección de comportamiento con reorganización

de estructura habría destruido la única afirmación que esta entrega puede sostener con evidencia, que el comportamiento preservado no cambió.

- **Las sobrecargas de compatibilidad tienen fecha de vencimiento.** Conservar las firmas antiguas permitió que la suite de Ae4 siguiera actuando como control independiente, pero deja dos vías de entrada equivalentes que habrá que retirar cuando ya no queden clientes de la versión anterior.

Respuesta a la pregunta central. El diseño mejoró de forma demostrable —cinco clases nuevas con una responsabilidad clara, reglas de negocio junto a los datos que gobiernan, tipos que declaran sus propias invariantes y ocho veces más pruebas—, y que el comportamiento no se alteró no es una afirmación de confianza sino un hecho verificable por tres vías independientes: 80 pruebas de caracterización verdes en los siete commits, sin una sola aserción modificada; la suite original de Ae4 intacta como control; y el harness de recibos de Ae4 re-ejecutado contra el código final produciendo una salida idéntica carácter por carácter. Cualquiera puede reproducir esa verificación commit por commit con los parches y los scripts incluidos.

15. Preguntas de defensa

¿Qué comportamiento protegiste antes de la primera refactorización?

Las validaciones de construcción de usuarios, horarios y reservas con sus mensajes exactos; todas las transiciones de estado válidas e inválidas y su efecto sobre el horario y las notas; los mensajes de cada rechazo del servicio, el orden de los eventos publicados a los observadores y la ausencia de efectos a medias cuando una operación se rechaza; y el texto completo de los seis recibos, carácter por carácter. Son 54 pruebas en un commit que no modifica código de producción.

¿Por qué seleccionaste esas seis refactorizaciones?

Por dos criterios explícitos: que el problema tuviera una consecuencia concreta sobre el mantenimiento — una clase con dos razones de cambio, un tipo que no declara su regla, una validación que otro cliente puede saltar, un grupo de parámetros que el compilador no puede proteger — y que estuviera cubierto por pruebas antes de tocarlo. Se ordenaron de menor a mayor riesgo: primero el Value Object local a una clase, después las extracciones de clase, y al final la que atraviesa la cadena de llamadas completa.

¿Qué prueba habría detectado una regresión concreta?

Tres ejemplos: si `Horario.reservar()` hubiera usado el mensaje de `marcarOcupado()` en lugar del del servicio, `solicitarUnHorarioOcupadoFallaConSuMensajeYSinPublicarEventos` habría fallado por el texto; si `componerNota` hubiera invertido el formato de los corchetes, `laObservacionDeLaPoliticaSeAnexaAlMotivoEntreCorchetes` lo habría marcado; y si `Dinero` hubiera cambiado el orden de la operación aritmética o el formato del importe, `reciboDeUnaReservaCompletadaConCodigoUeesAplicaDescuentoDeFidelidad` habría fallado por un total distinto de \$12.0.

¿Qué cambió en el diseño y qué permaneció igual funcionalmente?

Cambió la distribución de responsabilidades: cinco clases nuevas (`Correo`, `AgendaDocente`, `SolicitudTutoria`, `Dinero`, `TarifarioTutoria`), reglas movidas al objeto que posee el dato, condiciones con nombre y guard clauses antes de cualquier mutación. Permaneció igual todo lo observable: las mismas operaciones públicas, los mismos mensajes de error, las mismas transiciones de estado, los mismos eventos en el mismo orden y el mismo texto de recibo. La única diferencia de comportamiento es en caminos de error que antes dejaban el sistema a medias (un horario ocupado por una solicitud inválida), y está declarada y cubierta por pruebas.

¿Qué evidencia proporciona tu historial Git?

Que el proceso fue incremental y verificado: un commit por refactorización, ninguno que acumule dos, cada mensaje con el problema, el cambio, las pruebas que lo protegen y el resultado de la suite. Los logs de evidencia se regeneraron ejecutando la suite sobre cada commit del historial, y los commits están exportados como parches para reproducir la secuencia desde la línea base.

¿Qué costo o riesgo introdujo alguna de tus decisiones?

Tres, asumidos y documentados: las sobrecargas de compatibilidad en `ServicioReservas` y `Reserva` duplican la vía de entrada y habrá que retirarlas; `Dinero` conserva `double`, de modo que el riesgo de redondeo sigue presente hasta migrar a `BigDecimal` en un cambio funcional declarado; y el defecto latente de `publicarHorario` con nulo quedó sin corregir por disciplina de refactorización, lo que significa que sigue ahí hasta que se lo trate como el cambio de comportamiento que es.

16. Declaración de uso de inteligencia artificial

Durante el desarrollo de esta actividad utilicé herramientas de inteligencia artificial como apoyo. Las utilicé para: ampliar la suite hasta convertirla en una red de seguridad de caracterización a partir del comportamiento que yo definí como el que debía preservarse; redactar las clases y las pruebas de las seis refactorizaciones que seleccioné; construir el ejecutor de pruebas offline que permitió mantener el ciclo de verificación pese al bloqueo de red hacia Maven Central; y redactar la documentación de la entrega (tabla de refactorizaciones, comparación antes/después y este reporte).

La selección de los problemas de diseño a atacar, el orden en que se aplicaron las refactorizaciones, la decisión de conservar double en Dinero y la de no corregir el defecto latente detectado en publicarHorario son decisiones mías. Verifiqué el comportamiento ejecutando la suite completa antes y después de cada cambio, y puedo explicar y justificar cada línea del código, de las pruebas y de las conclusiones entregadas.

17. Enlace al repositorio y cómo reproducir

Elemento	Detalle
Repositorio	https://github.com/saaluis2242-droid/sistema-tutorias (rama main)
Ejecutar la suite	<code>mvn clean test</code> — o <code>bash tools/verificar.sh</code> para capturar el log como evidencia
Ejecutar la demostración	<code>mvn package && java -cp target/classes edu.uees.tutorias.app.Main</code>
Ver el historial completo	<code>git log --stat</code>
Reproducir paso a paso	<code>git checkout 5b9e3b1</code> y aplicar en orden los parches de <code>docs/ae5-evidencias/parches/</code> con <code>git am</code> , ejecutando la suite entre cada uno
Evidencias de la entrega	<code>docs/ae5-evidencias/</code> (suite verde por commit, código antes/después, diffs, git log, parches y ejecución de Main)

Fin del reporte.